# Agent Harness

企业决策者精简版

研究修订稿

修订日期（具体事实版本见各章）

2026-10-29

# 执行摘要

执行摘要 2
一、先区分四个结果 3
二、六个主案例的架构选择 4
三、分别决定谁运行、谁执行、谁留证 5
四、安全控制从具体动作开始 6
五、验证要看业务结果，也要看关键行为 7
六、多Agent按收益采用 8
七、自我进化需要可反驳的实验 9
八、从哪些地方开始 10

企业需要的不只是一个会回答的模型，而是能够在权限、预算和业务规则内完成工作的系统。Harness负责组织模型与工具的交互，管理上下文和执行状态；平台还要明确谁批准动作、谁保管凭证、谁确认业务完成，以及失败后怎么恢复。

建议先选定真实任务，再决定采购哪些能力。采购成品Agent、使用托管Harness、在企业环境运行开源运行时，都是可能的选择。先比较质量、集成和治理成本；只有数据表明某一层值得自己掌握时，再投入自研。保留任务、证据和退出路径的控制权，比默认采购多个供应商更重要。

# 一、先区分四个结果

模型停止生成、候选交付物通过检查、外部动作成功提交、业务目标验收通过，是不同事件。部署接口返回成功，但服务健康检查失败，应该记录“已提交、未完成”；取消请求被接受，也不能抹掉已发送的邮件或已提交的变更。

对于只需要一份回答或文件的任务，完成契约可以很轻。涉及生产系统、资金或敏感数据时，再增加独立验证、审批、对账和回滚机制。组件多不等于成熟，关键是机制与任务风险相匹配。

# 二、六个主案例的架构选择

| 案例 | 可借鉴的设计 | 企业要确认的边界 |
| --- | --- | --- |
| Claude Code / Agent SDK | 简洁循环、上下文、工具和生命周期扩展 | 与Claude Managed Agents托管服务分开选型；hook本身也有宿主执行权 |
| OpenAI Codex | 共享核心、App Server协议、CLI与SDK；新增Agents API托管接入 | 自管执行环境不等于自管会话存储，数据驻留与保留限制需单独核对 |
| Cursor | IDE上下文、云端代理、自管worker、Projects协调者 | worker可在企业网络，但推理与规划仍在云端；子智能体、worker与协调者承担不同职责 |
| DeepSeek Harness | 可组合插件与当前PTC进程执行 | 动态node:vm不是安全隔离；包安装、热重载和业务回滚不是同一个保证 |
| OpenHands | 当前SDK、Agent Server与执行工作区分离 | Canvas和SDK有不同版本；旧EventStream/Runtime架构属于历史实现 |
| pi-mono / Pi | 小循环核心、会话树与可组合扩展；v1.4.0 CLI 已内建 MCP | SDK须显式装配相关扩展；子智能体是扩展示例，运行权限和隔离由宿主承担 |

第三篇在2026-10-29改按统一问题讲解六种实现：先看组件和状态归属，再看循环、上下文、工具、安全、委派与验证。pi-mono固定于v1.4.0；仓库已迁至earendil-works/pi。Pi源码与职责、CLI与SDK装配差异

截至2026-09-19，dsh核验版本为0.1.6-alpha.2，Codex稳定版为0.155.1，OpenHands Canvas为1.20.0、SDK为1.49.2；这些是资料截面，不是永远适用的版本推荐。固定版本和迁移测试仍不可省略。dsh发布、Codex发布、OpenHands SDK发布

框架也没有退出舞台。Deep Agents的fork/isolated上下文模式、Microsoft Harness的组合式能力，说明企业可以复用一部分运行机制，再补足自己的业务验收。适合执行者的父上下文继承，不一定适合需要独立判断的审阅者。子代理上下文模式

# 三、分别决定谁运行、谁执行、谁留证

将职责拆开后，采购比较会更清楚：

| 决策 | 要确认的问题 |
| --- | --- |
| 模型循环 | 谁调度模型、管理上下文、响应执行中追加的指令？ |
| 工具执行 | 工具在哪台机器运行，能读写哪些资源，异常能否中止？ |
| 会话与证据 | 谁存原始事件、候选hash、批准与副作用记录，如何导出？ |
| 业务权威 | 谁能改变验收标准，谁能批准提交，谁处理未知结果？ |

按2026-09-19核验截面，OpenAI Agents API在9月10日进入公测，负责托管会话、编排、压缩和恢复；应用仍要选择工具与执行环境。当时文档明确自托管sandbox不使该API获得ZDR支持，本轮没有重验该服务的商业政策。Cursor的自管worker同样不把云端推理自动迁进企业网络。采购条款和数据流应据具体产品面核对。Agents API、Cursor自管机器

# 四、安全控制从具体动作开始

提示词中的“不访问生产”不是访问控制。批准应绑定动作、参数、资源、期限和契约版本，执行前再检查撤销和取消。企业应将子任务权限显式约束在父任务已获授权的范围内，并按产品核验工具过滤、会话权限与执行身份是否真正继承；如果需要领域专属身份，由有权主体独立签发，不能靠改一段子代理提示词扩权。

对结果未知的写动作，先对账。查询暂时没有记录不证明前次没生效；补偿接口存在也不意味着可以盲目重试。相同幂等键需要绑定同一操作和参数，下游的原子去重才提供相应保证。

插件、skill和hook要按版本治理。尤其hook可能绕过模型决策、直接由宿主执行，更新后的权限变化不能仅靠模型自检。持久记忆也要有来源、作用域和撤销传播，删除原条目不一定清除了派生摘要和规则。

# 五、验证要看业务结果，也要看关键行为

最终得分可以告诉团队结果好坏；行为检查帮助定位为什么。例如修改构建文件后有没有运行验证器，拒绝动作后有没有继续执行，候选封存后是否在干净环境重验。行为评测与端到端验收互补，不能规定所有复杂任务必须走唯一工具序列。Google行为评测实践

报告至少区分任务成功率、错误完成率、人工处理时间、每次业务验收成功成本、恢复时是否重复提交，以及关键任务切片退化。每个指标都要有观察窗口、分母和失败处理规则。零次观测到违规不是风险为零，统计不显著也不自动等于“不比原版差”。

# 六、多Agent按收益采用

适合并行、产物可独立验收、交接成本可控制的任务，可以委派。高度串行、频繁共享写状态或价值很低的任务，单代理可能更经济。父会话变短不等于总token或总费用减少，需要同时计算子任务重复探索和合并验证成本。

执行者可以继承已有调查上下文；审阅者通常只应拿任务、产物和验收材料，避免被父代理的自信结论锚定。工作区隔离后还要验证组合结果，各分支成功不能替代合并后的测试。

# 七、自我进化需要可反驳的实验

本书四类进化按改变对象划分：当前任务中的策略、跨任务经验、Harness实现与配置、模型参数。它们可以组合，也不是必须依次升级的台阶。另需判断谁决定改什么、谁提供反馈、谁作最终选择。

新研究提供了不同路线。HarnessDev评估创建和改进可运行Harness，结果显示稳定演化与跨模型迁移仍困难；HarnessEvolve用参考轨迹改进诊断，但需要正确答案；JIT-Agent训练生成Harness的辅助模型，不能因为执行模型冻结就说系统没有训练；SoL-Pi的效率配置省了token，却有平均得分让步。HarnessDev、HarnessEvolve、JIT-Agent、SoL-Pi

企业应把开发、候选选择与最终评价的数据用途分开。允许用于修复的反馈集不能继续冒充密封终测。失败、未激活和无返回的合格试验保留在主要成功率分母；生成、验证、失败重试和人工处置费用全部进入成本分子，重试不增加独立样本数。候选只能在授权范围内修改，评价器、根权限与发布控制另有版本和审批。紧急撤权优先于“让任务固定在旧版本”的便利。

# 八、从哪些地方开始

先选仓库修复、经营分析和一项高价值业务流程，定义完成契约和不可牺牲条件。用同一任务分布比较成品、托管与自管方案，记录适配、运行、验证、人工和退出成本。接着补最影响可靠性的权限、未知副作用处理和证据导出；再从真实失败建立回归集，决定是否值得替换具体运行组件。

本书提供python examples/run_examples.py作为本地教学入口：真实Git补丁和DST测试、固定数据快照的SQL分析、确定性进化门禁及安全控制反例。它验证这些明确条件下的实现逻辑，不是商业Agent评测，也不代替企业生产验收。

成熟度取决于承诺是否有相应证据，而不是是否拥有多个模型、多个代理或自动进化功能。每一项新增自主能力，都应有明确的预算、失败终态、审计入口和撤销方法。